1️⃣ Static Memory Allocation (Compile Time Memory)

🔹 কী?

যে মেমোরি **compile time-এ** allocate হয় এবং **program শেষ না হওয়া পর্যন্ত** থাকে।

🔹 কোথায় থাকে?

- **Stack**
- **Static / Data segment**

🔹 Static Memory এর ধরন

✅ (A) Local Static / Automatic Variable (Stack)

```
int a = 10;
```

- size fixed
- function শেষ হলে memory free
- fast access

✅ (B) Static Keyword ব্যবহার করলে

```
void counter() {
    static int count = 0;
    count++;
    cout << count << endl;
}
```

📌 Output:

```
1
2
3
```

✔ function শেষ হলেও memory free হয় না
✔ value preserve থাকে

🔹 Static Array

```
int arr[5] = {1,2,3,4,5};
```

✔ size fixed
❌ runtime এ size change করা যায় না

🔹 Static Memory এর বৈশিষ্ট্য

বিষয়	Static Memory
Allocation Time	Compile time
Size	Fixed
Speed	Fast
Memory Control	Compiler
Flexibility	❌ কম
Memory Waste	হতে পারে

🔹 Static Memory এর সমস্যা

❌ size change করা যায় না
❌ large data handle করা কঠিন
❌ memory waste হয়

2️⃣ Dynamic Memory Allocation (Runtime Memory)

🔹 কী?

যে মেমোরি **runtime-এ** allocate হয় এবং **programmer control করে**।

🔹 কোথায় থাকে?

- **Heap**

🔹 C++ Dynamic Memory Syntax

✅ Single Variable

```
int* ptr = new int;
*ptr = 10;
```

✅ Dynamic Array

```
int* arr = new int[5];
```

🔹 Memory Free করা (Very Important ⚠️)

```
delete ptr;
delete[] arr;
```

না করলে → **Memory Leak**

🔹 Dynamic Memory এর বৈশিষ্ট্য

বিষয়	Dynamic Memory
Allocation Time	Runtime
Size	Flexible
Speed	একটু Slow
Memory Control	Programmer
Flexibility	✅ বেশি
Memory Waste	কম

3️⃣ Static vs Dynamic Memory (Side by Side)

Feature	Static	Dynamic
Allocation	Compile Time	Runtime
Size Change	❌ No	✅ Yes

Feature	Static	Dynamic
Memory Area	Stack	Heap
Speed	Fast	Slower
Control	Compiler	Programmer
Use Case	Fixed data	Variable data

4️⃣ Real Life Example

🔹 Static Memory

👉 Class size fixed

```
int marks[5];
```

✔ Students = 5 always

🔹 Dynamic Memory

👉 User input অনুযায়ী

```
int n;
cin >> n;
int* marks = new int[n];
```

✔ Students count runtime এ আসে

5️⃣ Interview Important Points ⚠️

✔ Static memory faster
✔ Dynamic memory flexible
✔ new → allocate
✔ delete → free
✔ Heap memory manually manage করতে হয়

6️⃣ When to Use What?

✅ Static Memory ব্যবহার করো যখন:

- size আগে থেকেই জানা
- small data
- performance critical

✅ Dynamic Memory ব্যবহার করো যখন:

- runtime data
- large size
- user input dependent

7️⃣ Modern C++ Tip 🚀

❌ Avoid raw `new` / `delete`
✅ Use **STL Containers**

```
vector<int> v(n);
```

✔ safer
✔ no memory leak
✔ dynamic by default

How to declare a dynamic variable

1️⃣ Declare a Dynamic Variable (Single Variable)

🔹 Syntax

```
data_type* pointer = new data_type;
```

🔹 Example

```
int* p = new int;
*p = 10;

cout << *p << endl;  // Output: 10
```

🔹 Free the Memory (Important ⚠️)

```
delete p;
```

2️⃣ Declare a Dynamic Variable with Initialization

```
int* p = new int(20);
cout << *p << endl;  // 20
delete p;
```

3️⃣ Dynamic Array Declaration

🔹 Syntax

```
data_type* pointer = new data_type[size];
```

🔹 Example

```
int n;
cin >> n;

int* arr = new int[n];
```

🔹 Access

```
arr[0] = 5;
cout << arr[0];
```

🔹 Free Memory

```
delete[] arr;
```

4️⃣ Dynamic Variable using `malloc` (C-style – NOT recommended in C++)

```
int* p = (int*) malloc(sizeof(int));
*p = 30;
free(p);
```

⚠️ Avoid this in C++ (no constructor call)

5️⃣ Best Practice (Modern C++ ✅)

🔹 Use `std::unique_ptr`

```
#include <memory>

unique_ptr<int> p = make_unique<int>(50);
cout << *p << endl;
```

✔ automatic memory free
✔ no `delete` needed

🔹 Use `vector` instead of dynamic array

```
vector<int> v(n);
```

✔ safer
✔ resizable
✔ no memory leak

6️⃣ Summary Table

Method	Syntax	Memory Free
new	`int* p = new int;`	`delete`
new[]	`int* arr = new int[n];`	`delete[]`
unique_ptr	`make_unique<int>()`	Auto
vector	`vector<int>`	Auto

7️⃣ Interview Tip 🎯

👉 **Dynamic variable always declared using a pointer**
👉 Memory is allocated in **heap**
👉 Forgetting `delete` causes **memory leak**

🔹 Dynamic Array in C++ (Complete & Simple Guide)

A **dynamic array** is an array whose **size is decided at runtime** and memory is allocated in the **heap**.

1️⃣ Why Dynamic Array?

❌ Static array

```
int arr[5];   // size fixed
```

✅ Dynamic array

```
int n;
cin >> n;
int* arr = new int[n];   // size at runtime
```

2️⃣ Declaring a Dynamic Array

🔹 Syntax

```
data_type* array_name = new data_type[size];
```

🔹 Example

```
int n;
cin >> n;

int* arr = new int[n];
```

3️⃣ Initializing a Dynamic Array

🔹 Using Loop

```
for(int i = 0; i < n; i++) {
    arr[i] = i + 1;
```

```
}
```

🔹 Taking Input

```
for(int i = 0; i < n; i++) {
    cin >> arr[i];
}
```

4️⃣ Accessing Elements

```
cout << arr[0] << endl;
cout << arr[2] << endl;
```

✔ Same as normal array

5️⃣ Deleting a Dynamic Array (Very Important ⚠️)

```
delete[] arr;
```

❌ Wrong

```
delete arr;   // causes undefined behavior
```

6️⃣ Complete Example Program

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int* arr = new int[n];

    for(int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
```

```
    }

    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr;
    return 0;
}
```

7️⃣ Dynamic 2D Array (Bonus)

```
int rows, cols;
cin >> rows >> cols;

int** arr = new int*[rows];

for(int i = 0; i < rows; i++) {
    arr[i] = new int[cols];
}
```

🔹 Delete 2D Array

```
for(int i = 0; i < rows; i++) {
    delete[] arr[i];
}
delete[] arr;
```

8️⃣ Problems with Raw Dynamic Arrays ❌

- Memory leak risk
- Manual delete needed
- No automatic resizing

9️⃣ Best Practice (Modern C++ ✅)

🔹 Use `vector`

```
#include <vector>

vector<int> arr(n);
```

✔ safer
✔ auto memory management
✔ resizable

🔟 Static vs Dynamic Array (Quick Compare)

Feature	Static Array	Dynamic Array
Size	Fixed	Runtime
Memory	Stack	Heap
Resize	❌ No	✅ Yes
Safe	❌	✅ (vector)

🎯 Interview Key Points

✔ Dynamic arrays use **heap memory**
✔ `new[]` → allocate
✔ `delete[]` → deallocate
✔ Use `vector` in real projects

🔹 Returning a Dynamic Array from a Function in C++

When you create a **dynamic array inside a function**, you **can return it safely** because it lives in the **heap**, not on the stack.

1️⃣ Using `new[]` (Basic Method)

✅ Function

```
int* createArray(int n) {
    int* arr = new int[n];   // heap memory
```

```
    for(int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
    }
    return arr;   // returning address
}
```

✅ `main()`

```
int main() {
    int n = 5;
    int* arr = createArray(n);

    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr;   // VERY IMPORTANT
    return 0;
}
```

✔ Safe
✔ Common in exams
❌ Must manually `delete[]`

2️⃣ Why Static Array Cannot Be Returned ❌

```
int* func() {
    int arr[5];
    return arr;   // ❌ WRONG
}
```

❌ Stack memory destroyed after function ends
❌ Undefined behavior

3️⃣ Returning Dynamic Array using `vector` (BEST ✅)

✅ Function

```
#include <vector>
using namespace std;

vector<int> createArray(int n) {
```

```
    vector<int> v(n);
    for(int i = 0; i < n; i++) {
        v[i] = i + 1;
    }
    return v;   // safe, automatic
}
```

✅ `main()`

```
vector<int> arr = createArray(5);
```

✔ No memory leak
✔ Auto memory management
✔ Recommended in real projects

4️⃣ Using `unique_ptr` (Modern C++)

```
#include <memory>

unique_ptr<int[]> createArray(int n) {
    unique_ptr<int[]> arr(new int[n]);

    for(int i = 0; i < n; i++) {
        arr[i] = i * 2;
    }
    return arr;
}
```

✔ Ownership safety
✔ Auto delete

5️⃣ Summary Table

Method	Safe	Manual Delete	Recommended
`new[]` + pointer	✅	❌ Yes	⚠️ Only for learning
`static array`	❌	❌	❌
`vector`	✅	❌	⭐⭐⭐
`unique_ptr`	✅	❌	⭐⭐

🎯 Interview Key Points

✔ Heap memory can be returned
✔ Stack memory cannot be returned
✔ Caller is responsible for `delete[]`
✔ Prefer `vector` in modern C++

🔹 Returning a Dynamic 2D Array from a Function in C++

There are **3 correct ways** to return a dynamic 2D array.
I'll show **basic** → **modern** → **best practice**, with **clear examples**.

1️⃣ Using `int**` (Classic / Exam Method)

✅ Function

```
int** create2DArray(int rows, int cols) {
    int** arr = new int*[rows];

    for(int i = 0; i < rows; i++) {
        arr[i] = new int[cols];
    }

    // Initialize
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            arr[i][j] = i + j;
        }
    }

    return arr;   // heap memory
}
```

✅ `main()`

```
int main() {
    int r = 3, c = 4;
    int** arr = create2DArray(r, c);

    for(int i = 0; i < r; i++) {
```

```
        for(int j = 0; j < c; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    // Free memory
    for(int i = 0; i < r; i++) {
        delete[] arr[i];
    }
    delete[] arr;

    return 0;
}
```

⚠️ Drawbacks

- Manual delete
- Risk of memory leak
- Fragmented memory

2️⃣ Using Single Block Allocation (Better Performance)

✅ Function

```
int* create2DArray(int rows, int cols) {
    int* arr = new int[rows * cols];

    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            arr[i * cols + j] = i + j;
        }
    }

    return arr;
}
```

✅ Access

```
int* arr = create2DArray(r, c);
cout << arr[i * c + j];
```

✅ Delete

```
delete[] arr;
```

✔ Contiguous memory
✔ Faster
❌ Index calculation needed

3️⃣ Using `vector<vector<int>>` (BEST & RECOMMENDED ✅)

✅ Function

```
#include <vector>
using namespace std;

vector<vector<int>> create2DArray(int rows, int cols) {
    vector<vector<int>> arr(rows, vector<int>(cols));

    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            arr[i][j] = i * j;
        }
    }

    return arr;
}
```

✅ `main()`

```
vector<vector<int>> arr = create2DArray(3, 4);
```

✔ No delete needed
✔ Safe
✔ Clean code

4️⃣ Using `unique_ptr` (Modern C++)

```
#include <memory>

unique_ptr<int[]> create2DArray(int rows, int cols) {
    unique_ptr<int[]> arr(new int[rows * cols]);
```

```
    return arr;
}
```

✔ Auto delete
✔ Ownership safety

5️⃣ Comparison Table

Method	Safe	Easy	Performance	Recommended
`int**`	⚠️	❌	❌	❌ (learning only)
1D block	✅	⚠️	✅	⭐⭐
`vector<vector<int>>`	✅	✅	⚠️	⭐⭐⭐
`unique_ptr`	✅	⚠️	✅	⭐⭐

🎯 Interview Key Points

✔ Heap memory can be returned
✔ Stack memory cannot be returned
✔ Caller must delete heap memory
✔ Prefer STL containers

🔥 Common Interview Mistake ❌

```
int arr[3][4];
return arr;   // ❌ WRONG (stack memory)
```

🔹 How to Increase the Size of a Dynamic Array in C++

In **raw C++ dynamic arrays (`new[]`)**, you **CANNOT directly resize** an array.
You must **create a new bigger array and copy data**.

1️⃣ Manual Way (Exam / Concept Method)

✅ Steps

1. Create a new bigger array
2. Copy old elements
3. Delete old array
4. Point to new array

✅ Example: Increase Size

```
#include <iostream>
using namespace std;

int main() {
    int size = 5;
    int* arr = new int[size];

    for(int i = 0; i < size; i++) {
        arr[i] = i + 1;
    }

    // Increase size
    int newSize = 10;
    int* temp = new int[newSize];

    // Copy old data
    for(int i = 0; i < size; i++) {
        temp[i] = arr[i];
    }

    // Initialize new elements
    for(int i = size; i < newSize; i++) {
        temp[i] = 0;
    }

    delete[] arr;   // free old memory
    arr = temp;     // point to new array
    size = newSize;

    // Print
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr;
```

```
    return 0;
}
```

2️⃣ Why Direct Resize is Impossible ❌

```
arr = new int[10];   // ❌ old data lost + memory leak
```

Heap memory is **not contiguous by default** → resizing in-place is impossible.

3️⃣ Using `realloc()` (NOT recommended in C++ ❌)

```
arr = (int*) realloc(arr, newSize * sizeof(int));
```

❌ No constructor call
❌ Unsafe in C++
✔ Only C-style

4️⃣ BEST & PROFESSIONAL WAY ✅ → `vector`

```
#include <vector>
using namespace std;

vector<int> arr = {1,2,3,4,5};
arr.resize(10);
```

✔ Automatic memory management
✔ Safe
✔ Fast
✔ Industry standard

5️⃣ Dynamic Array Resize using Function

```
int* resizeArray(int* arr, int oldSize, int newSize) {
    int* temp = new int[newSize];

    for(int i = 0; i < oldSize; i++) {
```

```
        temp[i] = arr[i];
    }

    delete[] arr;
    return temp;
}
```

Usage:

```
arr = resizeArray(arr, oldSize, newSize);
```

6️⃣ Comparison Table

Method	Safe	Easy	Recommended
Manual copy (`new[]`)	⚠️	❌	Learning only
`realloc()`	❌	❌	❌
`vector`	✅	✅	⭐⭐⭐

🎯 Interview Key Points

✔ Dynamic arrays cannot grow directly
✔ Need new allocation + copy
✔ Must `delete[]` old memory
✔ Prefer `vector` in real projects

🔥 Common Mistakes When Resizing a Dynamic Array in C++

These mistakes are **VERY common in exams, interviews, and real code**.
I'll show **wrong code → why it's wrong → correct way**.

❌ Mistake 1: Directly Reassigning `new[]` (Memory Leak)

❌ Wrong

```
int* arr = new int[5];
arr = new int[10];   // ❌ memory leak
```

❌ Why?

- Old 5-element array is **lost**
- No `delete[]` → memory leak

✅ Correct

```
int* temp = new int[10];
copy(arr, arr + 5, temp);
delete[] arr;
arr = temp;
```

❌ Mistake 2: Forgetting to Delete Old Array

```
int* temp = new int[newSize];
// copy...
arr = temp;   // ❌ old memory never freed
```

✅ Correct

```
delete[] arr;
arr = temp;
```

❌ Mistake 3: Using `delete` Instead of `delete[]`

❌ Wrong

```
delete arr;   // ❌ undefined behavior
```

✅ Correct

```
delete[] arr;
```

❌ Mistake 4: Copying More Elements Than Old Size

```
for(int i = 0; i < newSize; i++) {
    temp[i] = arr[i];   // ❌ out of bounds
}
```

✅ Correct

```
for(int i = 0; i < oldSize; i++) {
    temp[i] = arr[i];
}
```

❌ Mistake 5: Losing the Size Information

```
int* arr = new int[5];
// resize...
// ❌ size variable not updated
```

✅ Correct

```
size = newSize;
```

❌ Mistake 6: Using `realloc()` in C++

```
arr = (int*) realloc(arr, newSize * sizeof(int));
```

❌ No constructor call
❌ Breaks C++ object safety
✔ Only valid in C

❌ Mistake 7: Returning Stack Array Instead of Heap

```
int* resize() {
    int temp[10];
    return temp;   // ❌ destroyed after function
}
```

✅ Correct

```
int* temp = new int[10];
return temp;
```

❌ Mistake 8: Forgetting to Initialize New Elements

```
// new elements contain garbage values
```

✅ Correct

```
for(int i = oldSize; i < newSize; i++) {
    temp[i] = 0;
}
```

❌ Mistake 9: Double Delete

```
delete[] arr;
delete[] arr;   // ❌ crash
```

✅ Correct

```
delete[] arr;
arr = nullptr;
```

❌ Mistake 10: Not Using `vector` When Allowed 😄

```
int* arr = new int[size];
```

✅ Best Practice

```
vector<int> arr;
arr.resize(newSize);
```

🧠 Golden Rule (Interview ⭐)

> **If you need resizing → use `vector`**

📌 Summary Table

Mistake	Result
No delete	Memory leak
Wrong delete	Undefined behavior
Wrong copy size	Crash
Stack return	Dangling pointer
realloc in C++	Unsafe

---

snieppet generator and this snieppet add in vs code.

Snippet Generator